

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: _____

Obtained Marks: _____

Object Oriented

Programming(lab)

ProjectProposal

(Bank Management System)

Submitted To:	Mr. Muhammad Hassan Mujtaba
Student Name:	M.TalalKhalid M.RehanKhan Ahmad Raza
Registration Number:	25108324 25108320 25108288

Table of Contents:

1. Introduction.....	1
2. Problem Statement	2
3. History & Background	3
4. Project Objectives	4
5. System Requirements.....	5
6. Technical Specifications	6
7. Data Structures Used.....	7
8. System Architecture	8
9. Core Features & Functionality	9
10.Implementation Plan	10
11.Advantages.....	11
12.Disadvantages	12
13.Limitations	13
14.Testing Strategy	14
15.Methodology.....	15
16.Expected Outcomes	16
17.Flowchart	16
18.Overall Methodology Diagram.....	17
19.Future Enhancements.....	18
20.Conclusion	19
21.References.....	20

ProjectTitle

Bank Management System

❖ Introduction:

The Bank Management System is a console-based C++ application designed to manage basic banking operations such as account creation, deposit, withdrawal, and balance inquiry. This project demonstrates core **Object-Oriented Programming (OOP)** concepts including **classes, objects, inheritance, encapsulation, polymorphism, file handling, and abstraction**. It provides a simple and interactive system where users can perform banking tasks efficiently through a text-based interface.

❖ Problem Statement:

Traditional manual banking systems are time-consuming, error-prone, and inefficient for managing customer records and transactions. There is a need for a simple computerized system that can automate banking operations, ensure accuracy, and provide quick access to account information using programming concepts.

❖ History & Background:

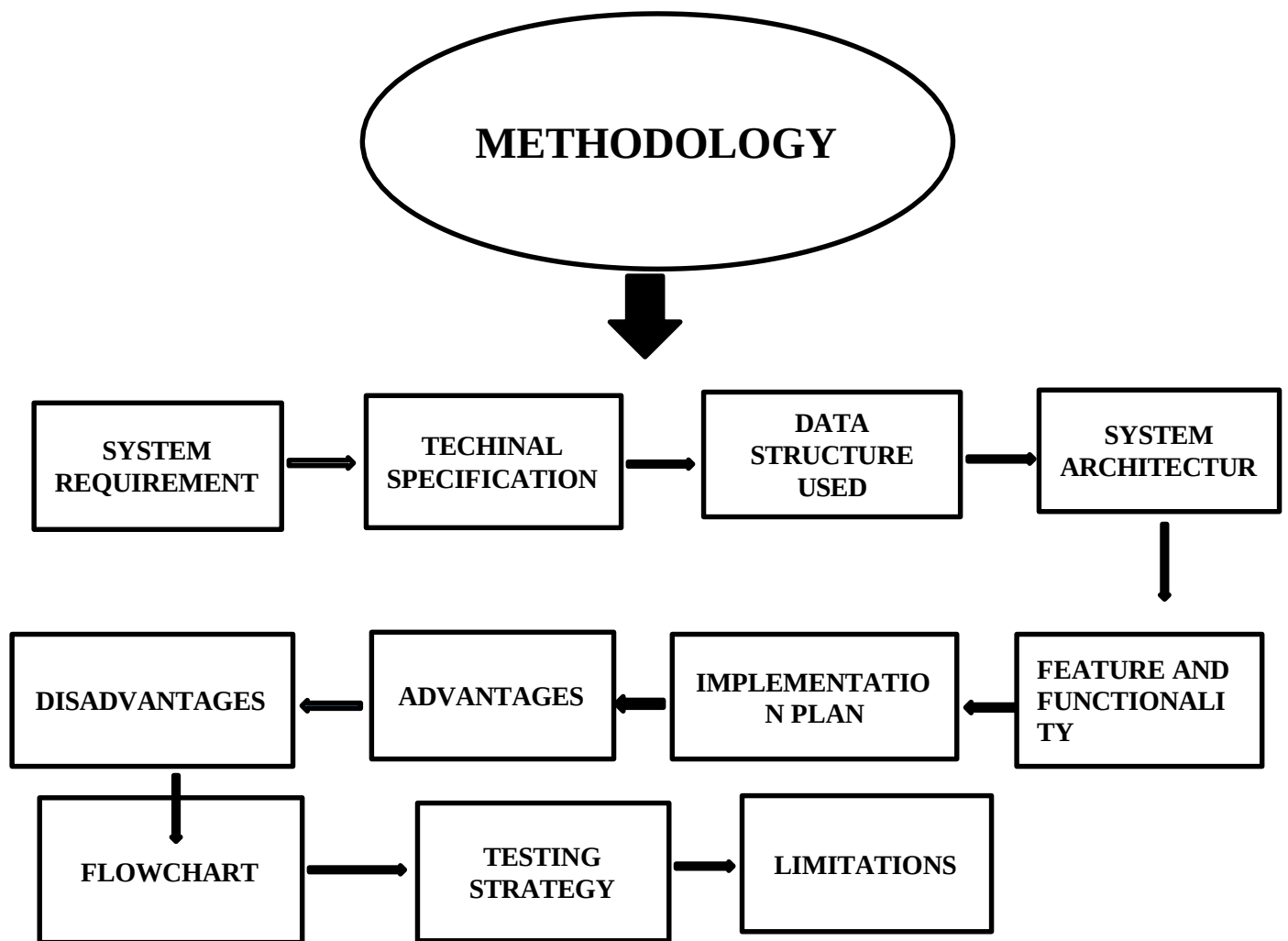
Banking systems have evolved from manual record-keeping to fully automated digital systems. Early systems were simple text-based programs that handled customer data. With the advancement of programming, especially OOP in C++, developers began creating modular and scalable banking applications. This project represents a basic implementation of such systems using OOP principles.

❖ Project Objectives:

- To develop a console-based banking system using C++ OOP concepts
- To implement classes and objects for account management
- To apply inheritance for different account types (Saving/Current)
- To use encapsulation to secure account data
- To implement file handling for storing customer records
- To demonstrate real-world application of OOP concepts

❖ Methodology:

The project follows a **Structured Programming Approach**—using step-by-step modular development with functions and flow control.



1. SystemRequirements:

• Software:

- OperatingSystem:Windows10orlater
- IDE:Dev-C++/Code::Blocks/VisualStudio
- Compiler:GCCorMSVC

2. TechnicalSpecifications:

- Language: C++
- Input/Output: Console (text-based)
- Concepts Used:
- Classes & Objects
- Inheritance
- Encapsulation
- Polymorphism
- File Handling
- Functions & Control Structures

3. DataStructuresUsed:

- Classes — To represent bank accounts and users
- Objects — To create multiple account instances
- Arrays / Vectors — To store multiple accounts
- File Streams — To save and retrieve account data

4. SystemArchitecture:

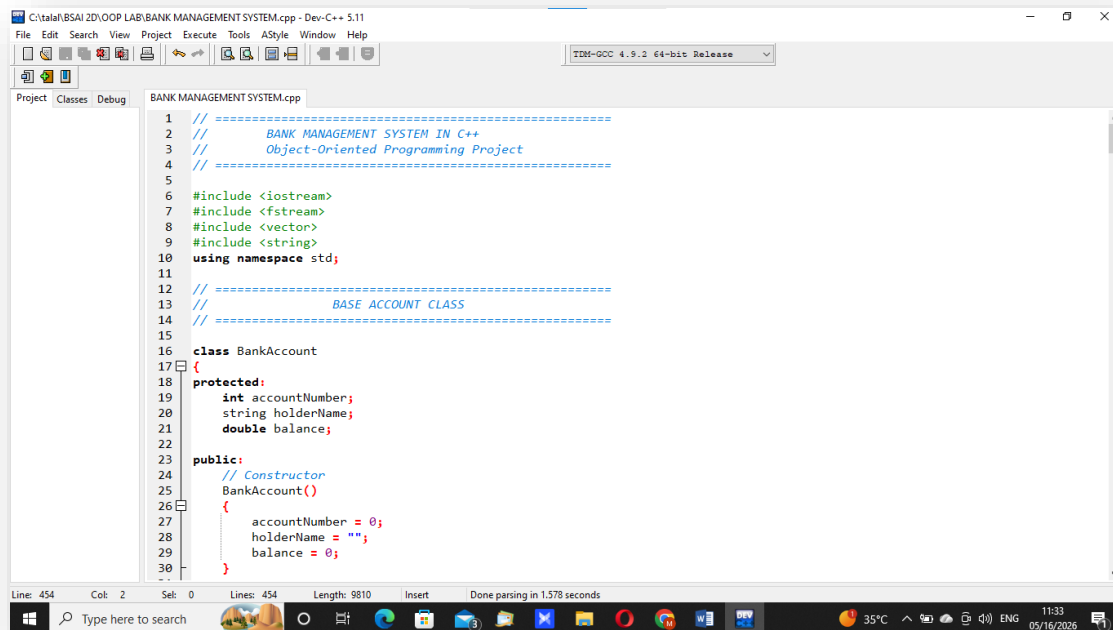
The system follows a modular OOP structure:

- **User Interface Module:** Takes input from user
- **Account Class Module:** Handles account data and operations
- **Transaction Module:** Performs deposit and withdrawal
- **File Handling Module:** Stores and retrieves data
- **Admin Module (optional):** Manages all accounts

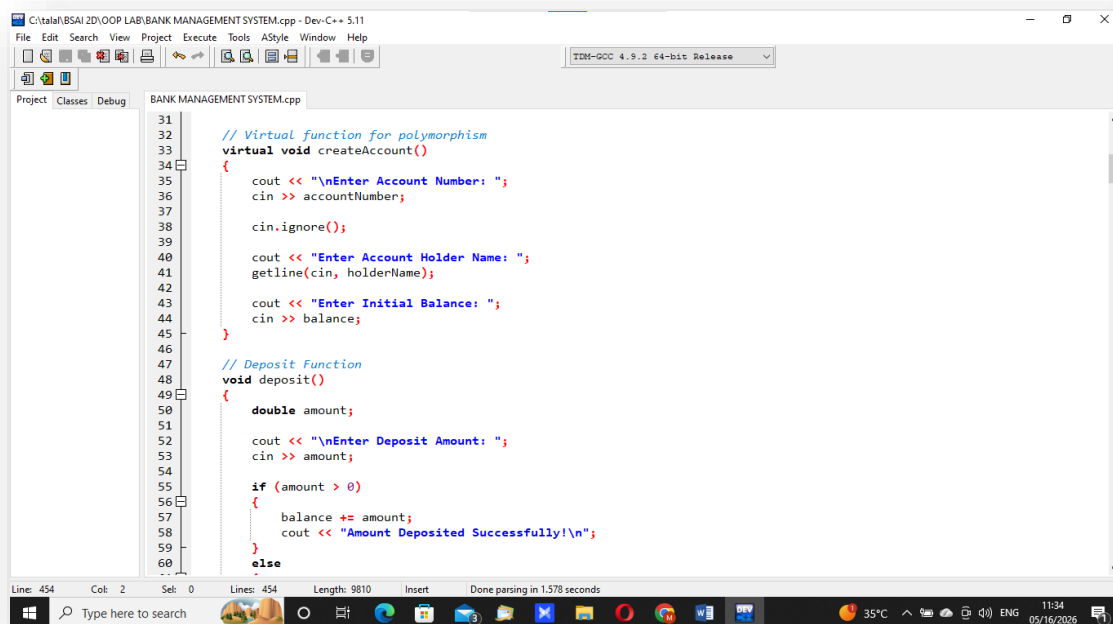
5. CoreFeatures&Functionality:

- Create new bank account
- Deposit money
- Withdraw money
- Check account balance
- Display account details
- Save and load data using files

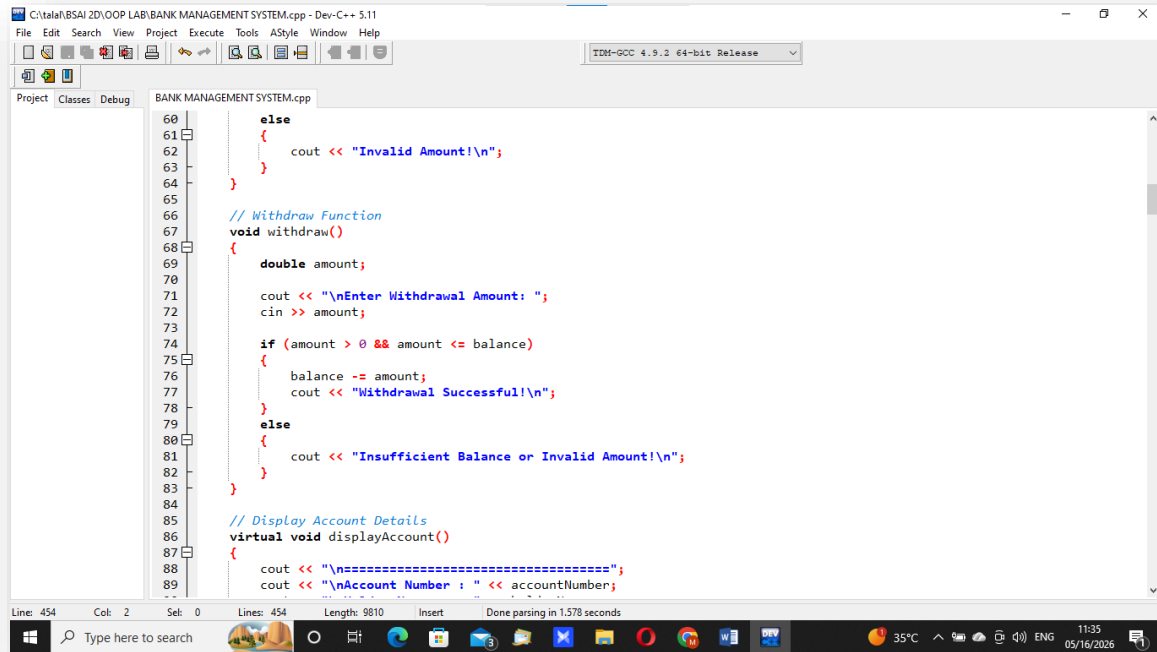
6. Implementation CODE:



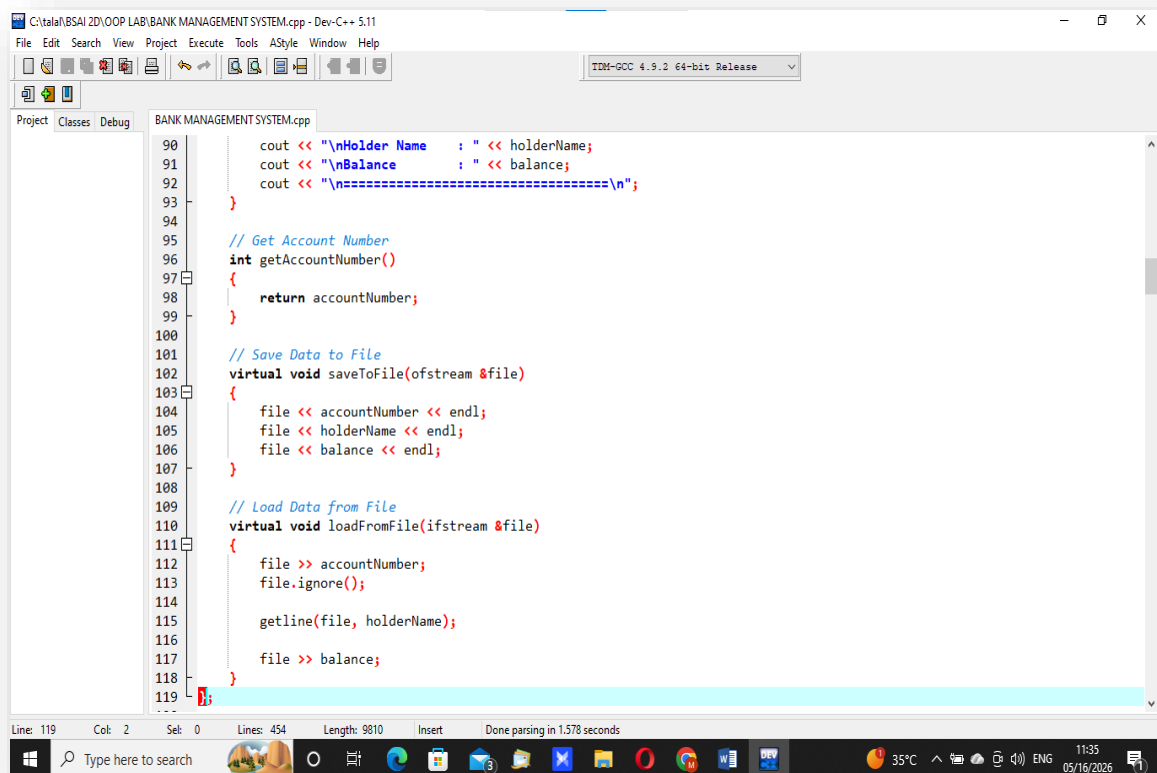
```
1 // =====
2 //      BANK MANAGEMENT SYSTEM IN C++
3 //      Object-Oriented Programming Project
4 // =====
5
6 #include <iostream>
7 #include <fstream>
8 #include <vector>
9 #include <string>
10 using namespace std;
11
12 // =====
13 //      BASE ACCOUNT CLASS
14 // =====
15
16 class BankAccount
17 {
18 protected:
19     int accountNumber;
20     string holderName;
21     double balance;
22
23 public:
24     // Constructor
25     BankAccount()
26     {
27         accountNumber = 0;
28         holderName = "";
29         balance = 0;
30     }
31 }
```



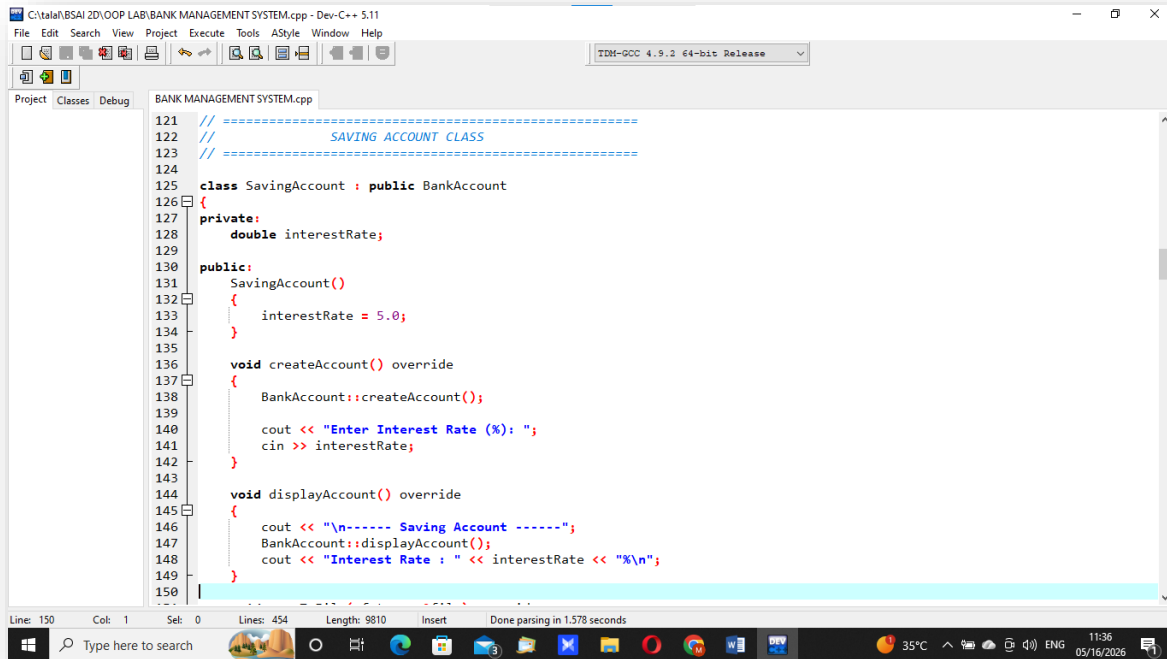
```
31
32 // Virtual function for polymorphism
33 virtual void createAccount()
34 {
35     cout << "\nEnter Account Number: ";
36     cin >> accountNumber;
37
38     cin.ignore();
39
40     cout << "Enter Account Holder Name: ";
41     getline(cin, holderName);
42
43     cout << "Enter Initial Balance: ";
44     cin >> balance;
45 }
46
47 // Deposit Function
48 void deposit()
49 {
50     double amount;
51
52     cout << "\nEnter Deposit Amount: ";
53     cin >> amount;
54
55     if (amount > 0)
56     {
57         balance += amount;
58         cout << "Amount Deposited Successfully!\n";
59     }
60     else
61     {
62         cout << "Invalid Amount!\n";
63     }
64 }
```



```
60         else
61         {
62             cout << "Invalid Amount!\n";
63         }
64     }
65
66     // Withdraw Function
67     void withdraw()
68     {
69         double amount;
70
71         cout << "\nEnter Withdrawal Amount: ";
72         cin >> amount;
73
74         if (amount > 0 && amount <= balance)
75         {
76             balance -= amount;
77             cout << "Withdrawal Successful!\n";
78         }
79         else
80         {
81             cout << "Insufficient Balance or Invalid Amount!\n";
82         }
83     }
84
85     // Display Account Details
86     virtual void displayAccount()
87     {
88         cout << "\n=====";
89         cout << "\nAccount Number : " << accountNumber;
```

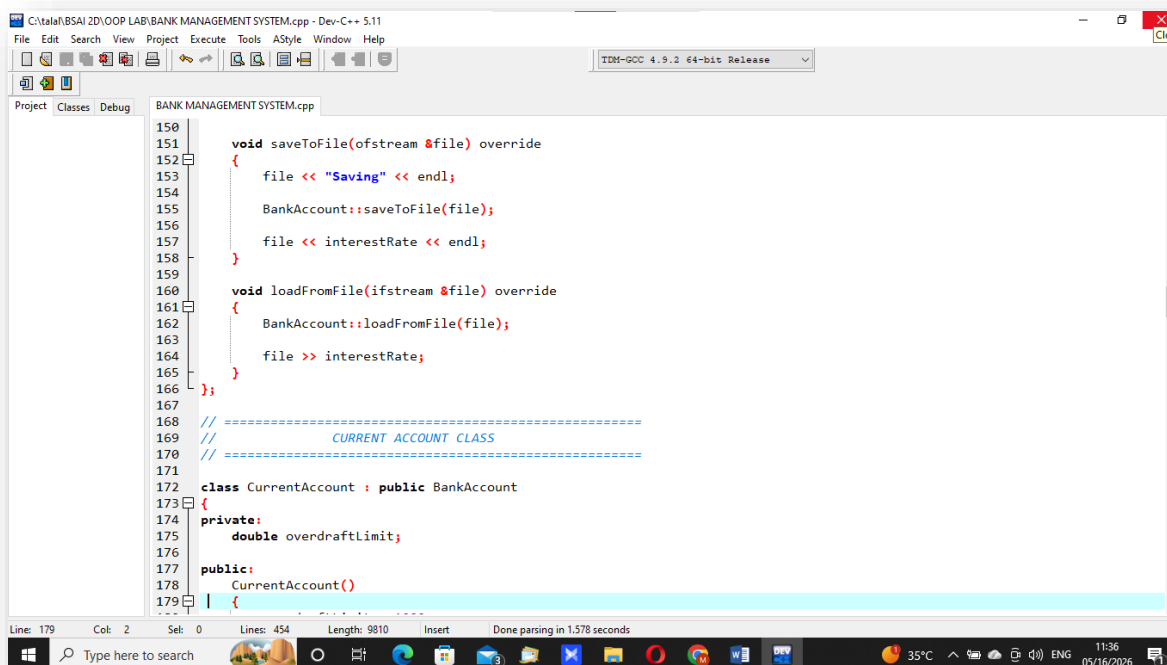


```
90         cout << "\nHolder Name   : " << holderName;
91         cout << "\nBalance       : " << balance;
92         cout << "\n=====";
93     }
94
95     // Get Account Number
96     int getAccountNumber()
97     {
98         return accountNumber;
99     }
100
101     // Save Data to File
102     virtual void saveToFile(ofstream &file)
103     {
104         file << accountNumber << endl;
105         file << holderName << endl;
106         file << balance << endl;
107     }
108
109     // Load Data from File
110     virtual void loadFromFile(istream &file)
111     {
112         file >> accountNumber;
113         file.ignore();
114
115         getline(file, holderName);
116
117         file >> balance;
118     }
119 }
```



The screenshot shows a C++ IDE window titled "C:\talah\BSAI\2D\OOP LAB\BANK MANAGEMENT SYSTEM.cpp - Dev-C++ 5.11". The code defines the `SavingAccount` class, which inherits from `BankAccount`. The class has a private member `interestRate` and public methods `SavingAccount()`, `createAccount()`, and `displayAccount()`. The `createAccount()` method prompts the user to enter an interest rate. The `displayAccount()` method displays the account details, including the interest rate. The status bar at the bottom indicates "Line: 150, Col: 1, Sel: 0, Lines: 454, Length: 9810, Insert, Done parsing in 1.578 seconds".

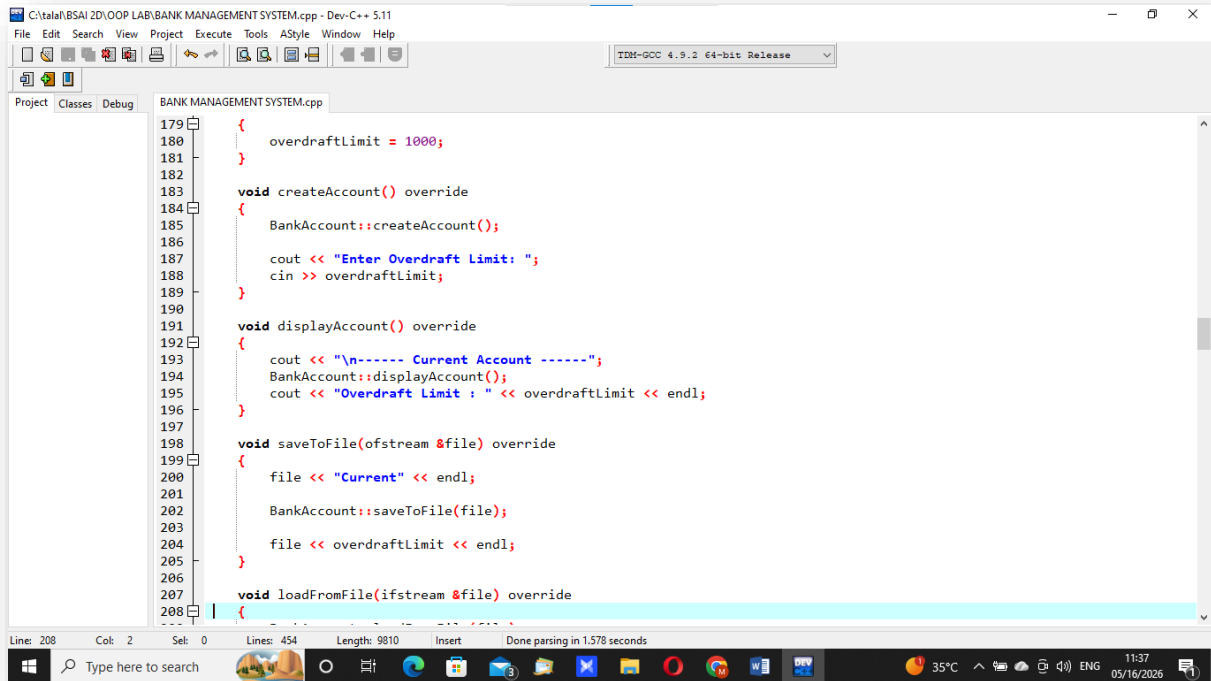
```
121 // =====  
122 // SAVING ACCOUNT CLASS  
123 // =====  
124  
125 class SavingAccount : public BankAccount  
126 {  
127 private:  
128     double interestRate;  
129  
130 public:  
131     SavingAccount()  
132     {  
133         interestRate = 5.0;  
134     }  
135  
136     void createAccount() override  
137     {  
138         BankAccount::createAccount();  
139  
140         cout << "Enter Interest Rate (%): ";  
141         cin >> interestRate;  
142     }  
143  
144     void displayAccount() override  
145     {  
146         cout << "\n----- Saving Account -----";  
147         BankAccount::displayAccount();  
148         cout << "Interest Rate : " << interestRate << "%\n";  
149     }  
150 }
```



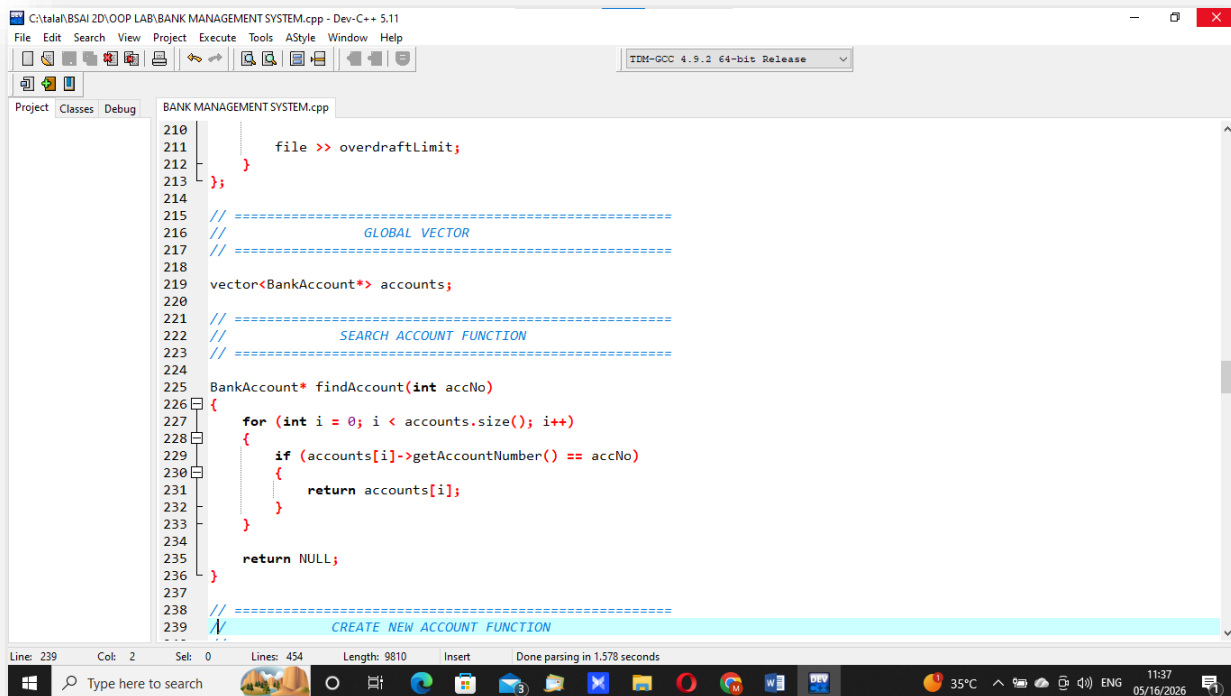
The screenshot shows the same C++ IDE window, now displaying the `CurrentAccount` class implementation. The class inherits from `BankAccount` and has a private member `overdraftLimit`. It includes methods `saveToFile()` and `loadFromFile()` for file operations. The `saveToFile()` method writes the account details to a file, and the `loadFromFile()` method reads the details from a file. The status bar at the bottom indicates "Line: 179, Col: 2, Sel: 0, Lines: 454, Length: 9810, Insert, Done parsing in 1.578 seconds".

```
150  
151  
152 void saveToFile(ofstream &file) override  
153 {  
154     file << "Saving" << endl;  
155     BankAccount::saveToFile(file);  
156  
157     file << interestRate << endl;  
158 }  
159  
160 void loadFromFile(istream &file) override  
161 {  
162     BankAccount::loadFromFile(file);  
163  
164     file >> interestRate;  
165 }  
166  
167  
168 // =====  
169 // CURRENT ACCOUNT CLASS  
170 // =====  
171  
172 class CurrentAccount : public BankAccount  
173 {  
174 private:  
175     double overdraftLimit;  
176  
177 public:  
178     CurrentAccount()  
179     {
```


ShaheedZulfikarAliBhuttoInstituteofScience&TechnologyUniversity

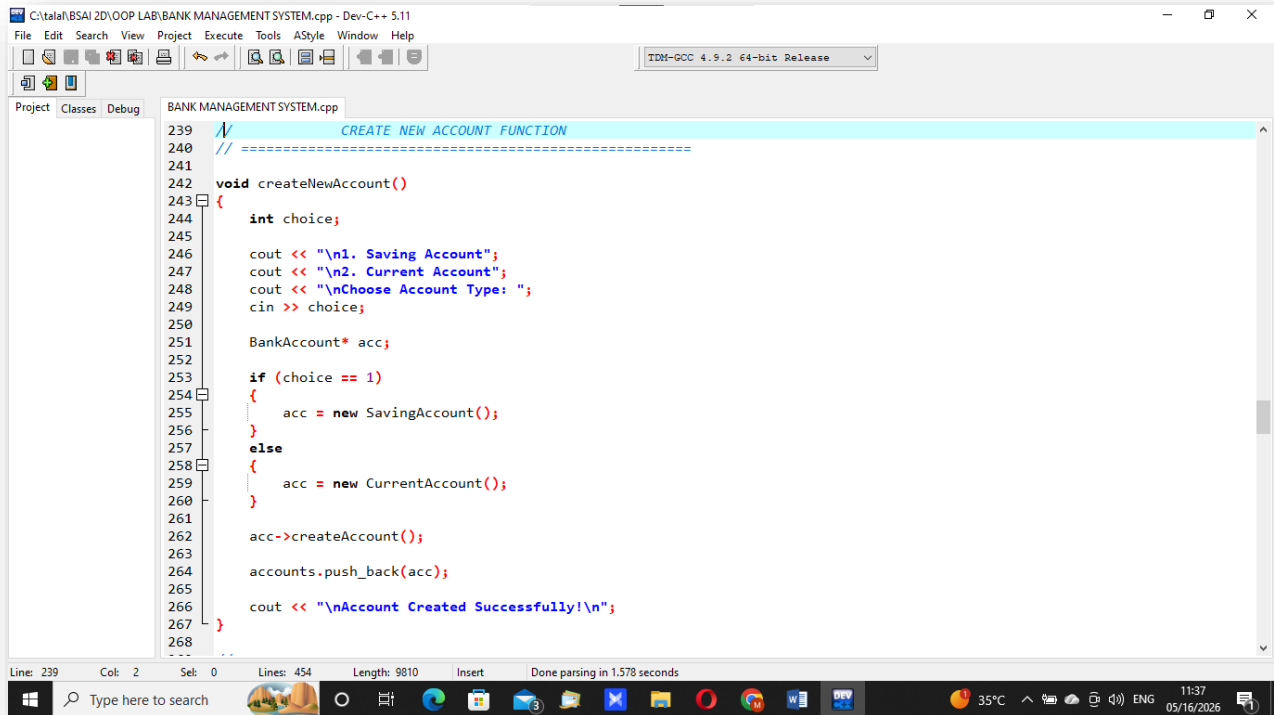


```
C:\Itala\BSAI 2D\OOP LAB\BANK MANAGEMENT SYSTEM.cpp - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
IDM-GCC 4.9.2 64-bit Release
Project Classes Debug BANK MANAGEMENT SYSTEM.cpp
179 {
180     overdraftLimit = 1000;
181 }
182
183 void createAccount() override
184 {
185     BankAccount::createAccount();
186
187     cout << "Enter Overdraft Limit: ";
188     cin >> overdraftLimit;
189 }
190
191 void displayAccount() override
192 {
193     cout << "\n----- Current Account -----";
194     BankAccount::displayAccount();
195     cout << "Overdraft Limit : " << overdraftLimit << endl;
196 }
197
198 void saveToFile(ofstream &file) override
199 {
200     file << "Current" << endl;
201
202     BankAccount::saveToFile(file);
203
204     file << overdraftLimit << endl;
205 }
206
207 void loadFromFile(istream &file) override
208 {
209 }
```



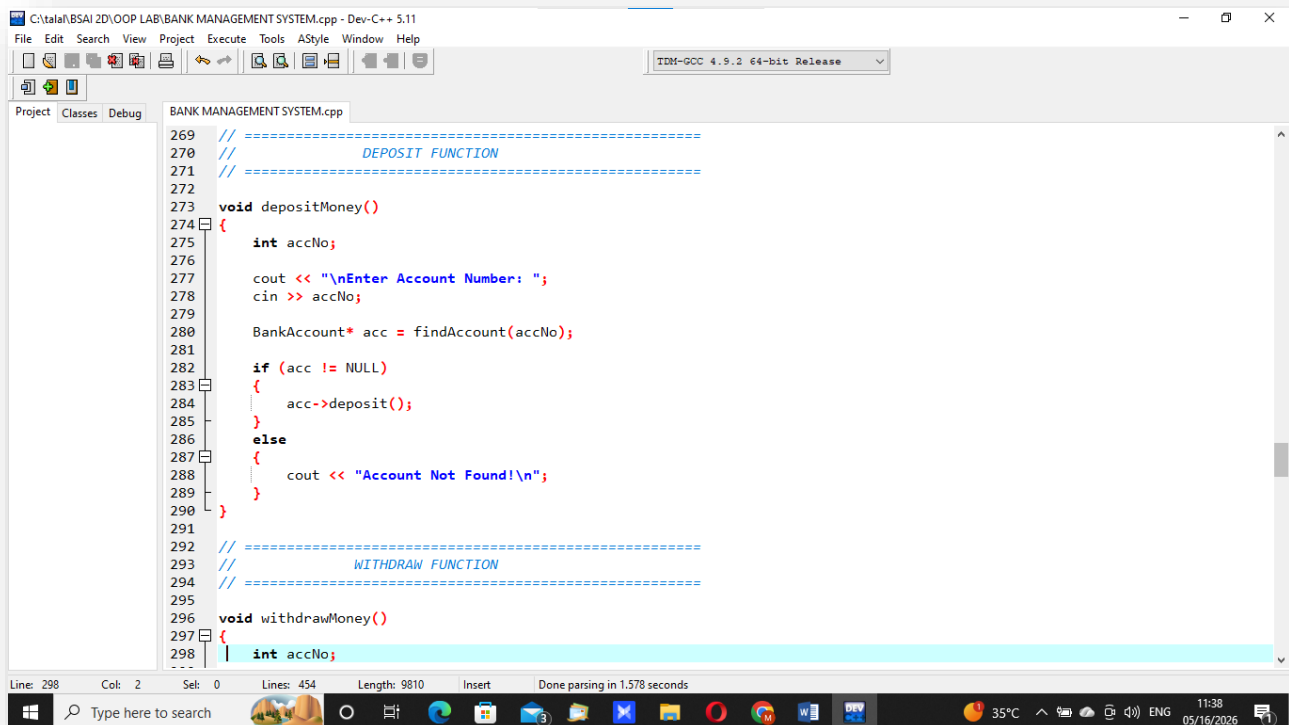
```
C:\Itala\BSAI 2D\OOP LAB\BANK MANAGEMENT SYSTEM.cpp - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
IDM-GCC 4.9.2 64-bit Release
Project Classes Debug BANK MANAGEMENT SYSTEM.cpp
210     file >> overdraftLimit;
211 }
212 }
213 };
214
215 // =====
216 // GLOBAL VECTOR
217 // =====
218
219 vector<BankAccount*> accounts;
220
221 // =====
222 // SEARCH ACCOUNT FUNCTION
223 // =====
224
225 BankAccount* findAccount(int accNo)
226 {
227     for (int i = 0; i < accounts.size(); i++)
228     {
229         if (accounts[i]->getAccountNumber() == accNo)
230         {
231             return accounts[i];
232         }
233     }
234
235     return NULL;
236 }
237
238 // =====
239 // CREATE NEW ACCOUNT FUNCTION
```

ShaheedZulfikarAliBhuttoInstituteofScience&TechnologyUniversity



The screenshot shows a C++ IDE with the file 'BANK MANAGEMENT SYSTEM.cpp' open. The code is for a 'CREATE NEW ACCOUNT FUNCTION'. It starts with a comment 'CREATE NEW ACCOUNT FUNCTION' and a series of slashes. The function 'createNewAccount()' is defined, which takes an integer 'choice' as input. It uses 'cout' to display options for 'Saving Account' and 'Current Account', and 'cin' to get the user's choice. Based on the choice, it creates a new 'SavingAccount' or 'CurrentAccount' object and pushes it into an 'accounts' vector. The function ends with a success message.

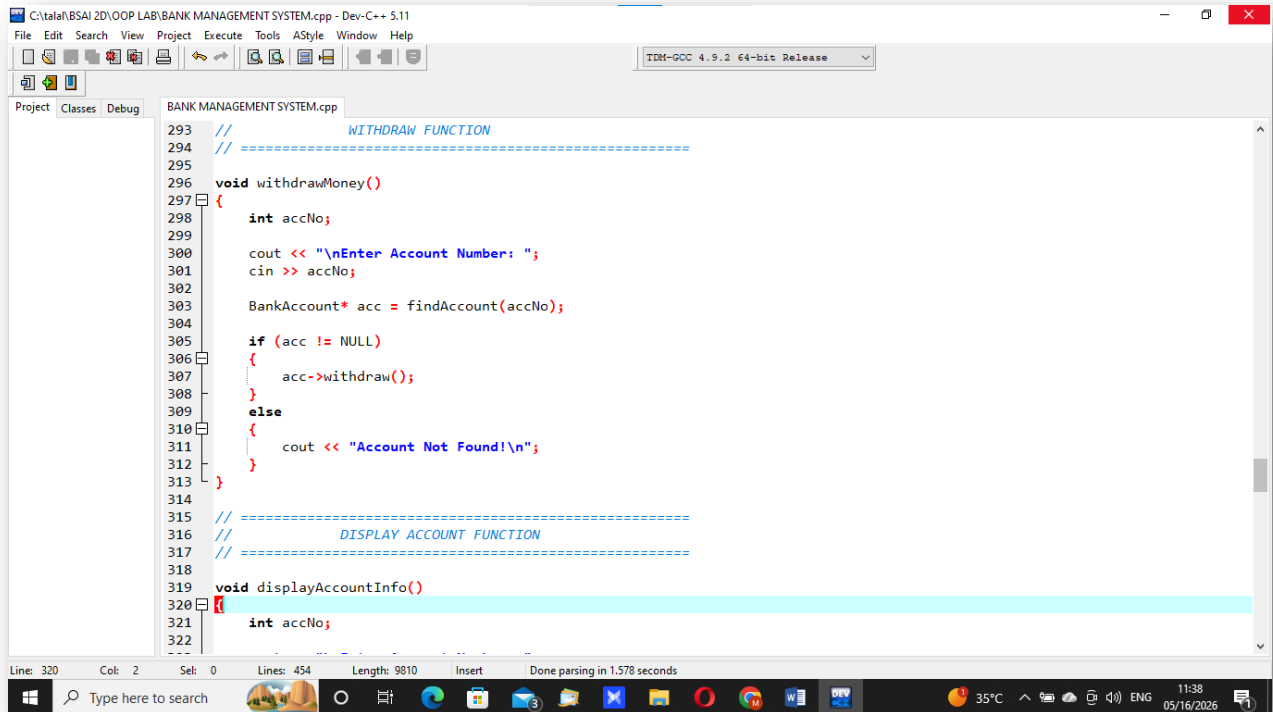
```
239 // CREATE NEW ACCOUNT FUNCTION
240 // =====
241
242 void createNewAccount()
243 {
244     int choice;
245
246     cout << "\n1. Saving Account";
247     cout << "\n2. Current Account";
248     cout << "\nChoose Account Type: ";
249     cin >> choice;
250
251     BankAccount* acc;
252
253     if (choice == 1)
254     {
255         acc = new SavingAccount();
256     }
257     else
258     {
259         acc = new CurrentAccount();
260     }
261
262     acc->createAccount();
263     accounts.push_back(acc);
264
265     cout << "\nAccount Created Successfully!\n";
266 }
267
268
```



The screenshot shows the same C++ IDE with the file 'BANK MANAGEMENT SYSTEM.cpp'. The code continues with a 'DEPOSIT FUNCTION' and the start of a 'WITHDRAW FUNCTION'. The 'depositMoney()' function takes an 'accNo' as input, finds the account, and calls 'deposit()' on it. The 'withdrawMoney()' function is partially visible, starting with an 'int accNo;' declaration.

```
269 // =====
270 // DEPOSIT FUNCTION
271 // =====
272
273 void depositMoney()
274 {
275     int accNo;
276
277     cout << "\nEnter Account Number: ";
278     cin >> accNo;
279
280     BankAccount* acc = findAccount(accNo);
281
282     if (acc != NULL)
283     {
284         acc->deposit();
285     }
286     else
287     {
288         cout << "Account Not Found!\n";
289     }
290 }
291
292 // =====
293 // WITHDRAW FUNCTION
294 // =====
295
296 void withdrawMoney()
297 {
298     int accNo;
299
```

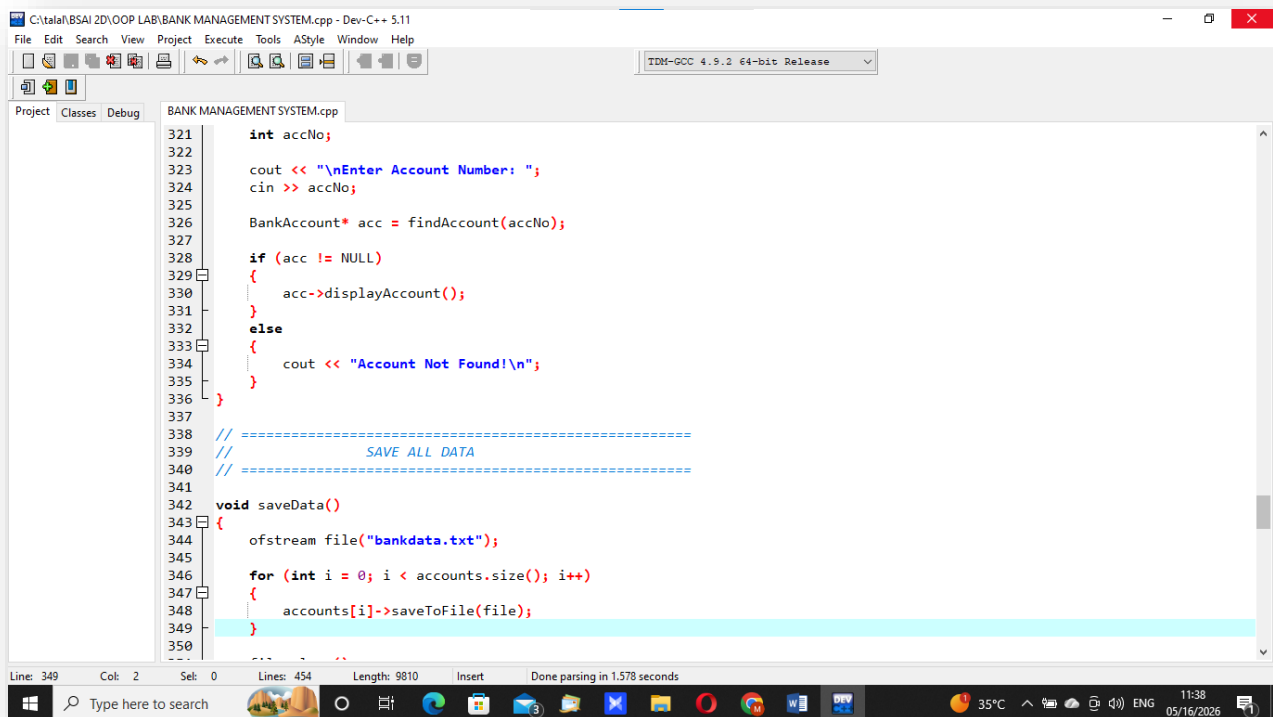
ShaheedZulfikarAliBhuttoInstituteofScience&TechnologyUniversity



The screenshot shows the Dev-C++ IDE with the file 'BANK MANAGEMENT SYSTEM.cpp'. The code is as follows:

```
293 // ===== WITHDRAW FUNCTION =====
294
295
296 void withdrawMoney()
297 {
298     int accNo;
299
300     cout << "\nEnter Account Number: ";
301     cin >> accNo;
302
303     BankAccount* acc = findAccount(accNo);
304
305     if (acc != NULL)
306     {
307         acc->withdraw();
308     }
309     else
310     {
311         cout << "Account Not Found!\n";
312     }
313 }
314
315 // ===== DISPLAY ACCOUNT FUNCTION =====
316
317
318 void displayAccountInfo()
319 {
320     int accNo;
```

The status bar at the bottom indicates 'Line: 320', 'Col: 2', 'Sel: 0', 'Lines: 454', 'Length: 9810', 'Insert', and 'Done parsing in 1.578 seconds'. The Windows taskbar shows the date as 05/16/2026 and time as 11:38.

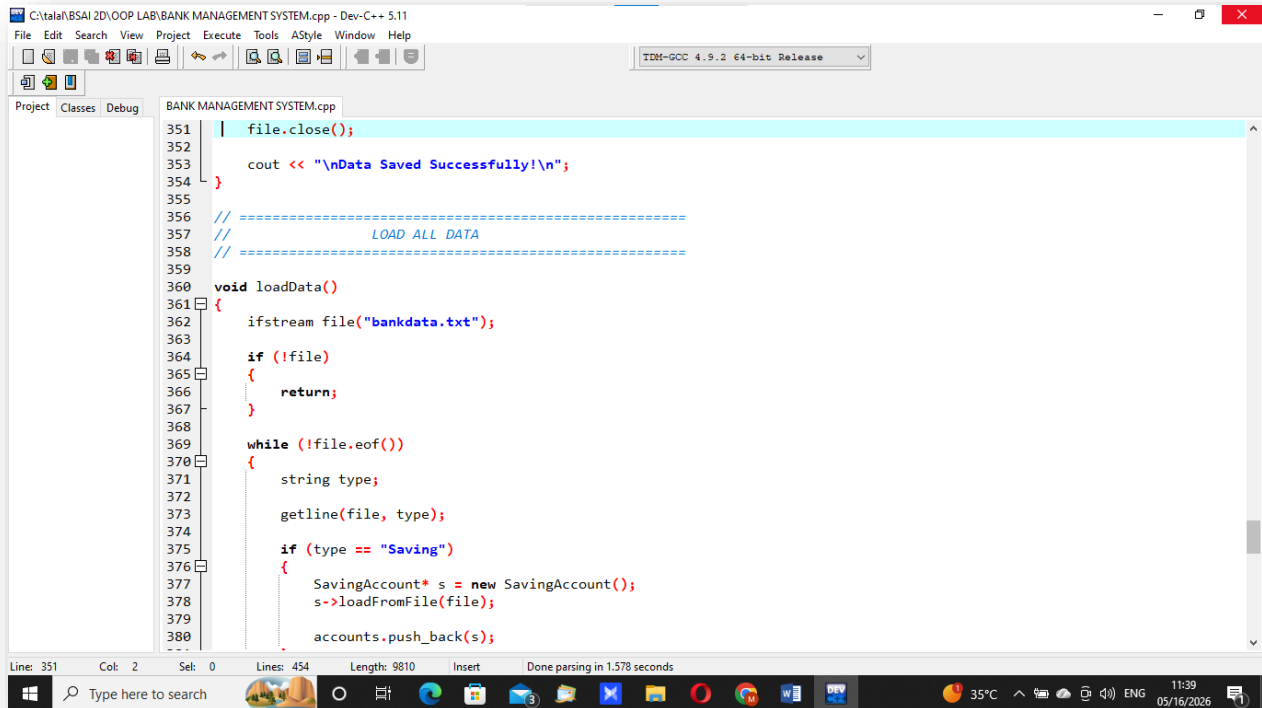


The screenshot shows the Dev-C++ IDE with the file 'BANK MANAGEMENT SYSTEM.cpp'. The code is as follows:

```
321 int accNo;
322
323 cout << "\nEnter Account Number: ";
324 cin >> accNo;
325
326 BankAccount* acc = findAccount(accNo);
327
328 if (acc != NULL)
329 {
330     acc->displayAccount();
331 }
332 else
333 {
334     cout << "Account Not Found!\n";
335 }
336
337 // ===== SAVE ALL DATA =====
338
339
340 void saveData()
341 {
342     ofstream file("bankdata.txt");
343
344     for (int i = 0; i < accounts.size(); i++)
345     {
346         accounts[i]->saveToFile(file);
347     }
348 }
349
350
```

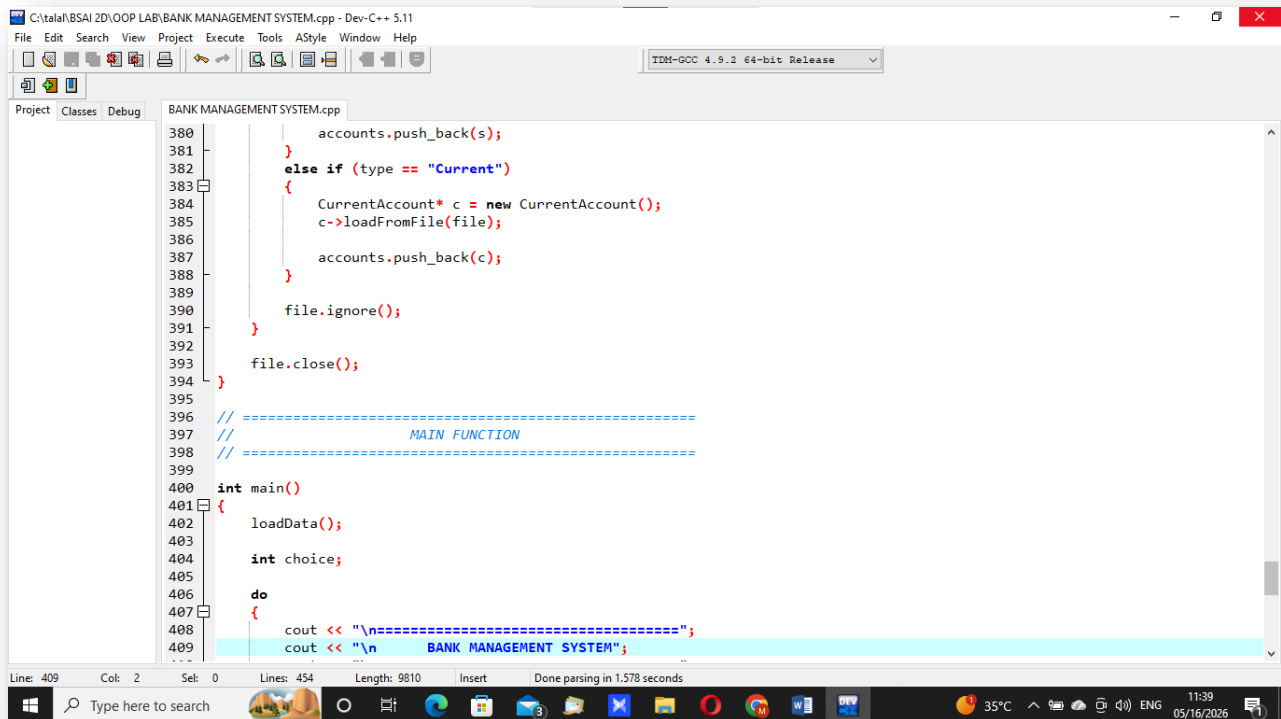
The status bar at the bottom indicates 'Line: 349', 'Col: 2', 'Sel: 0', 'Lines: 454', 'Length: 9810', 'Insert', and 'Done parsing in 1.578 seconds'. The Windows taskbar shows the date as 05/16/2026 and time as 11:38.

ShaheedZulfikarAliBhuttoInstituteofScience&TechnologyUniversity



The screenshot shows a C++ IDE with the file `BANK MANAGEMENT SYSTEM.cpp` open. The code is written in C++ and uses the `g++` compiler. The `loadData()` function is highlighted, showing it reads data from `bankdata.txt` and pushes it into a vector of accounts. The code is as follows:

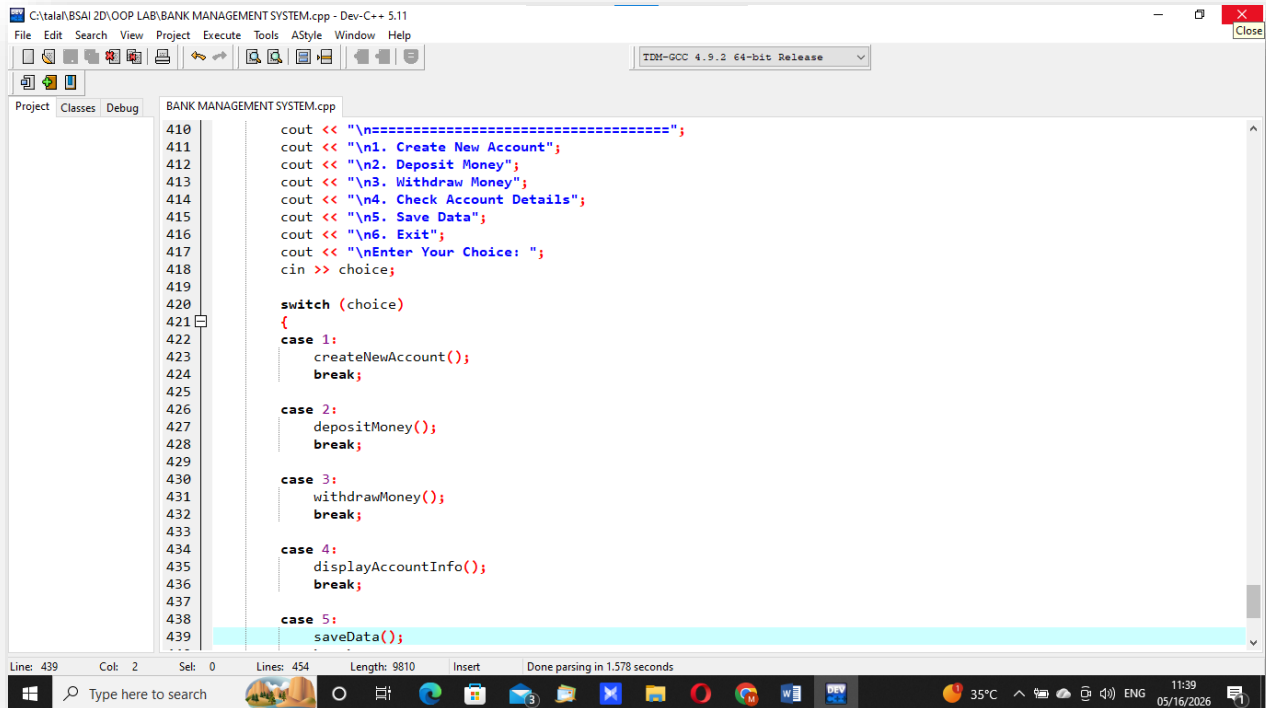
```
351 | file.close();
352 |
353 |     cout << "\nData Saved Successfully!\n";
354 | }
355 |
356 | // =====
357 | //          LOAD ALL DATA
358 | // =====
359 |
360 | void loadData()
361 | {
362 |     ifstream file("bankdata.txt");
363 |
364 |     if (!file)
365 |     {
366 |         return;
367 |     }
368 |
369 |     while (!file.eof())
370 |     {
371 |         string type;
372 |         getline(file, type);
373 |
374 |         if (type == "Saving")
375 |         {
376 |             SavingAccount* s = new SavingAccount();
377 |             s->loadFromFile(file);
378 |
379 |             accounts.push_back(s);
380 |         }
```



The screenshot shows the same C++ IDE with the `main()` function highlighted. The code is as follows:

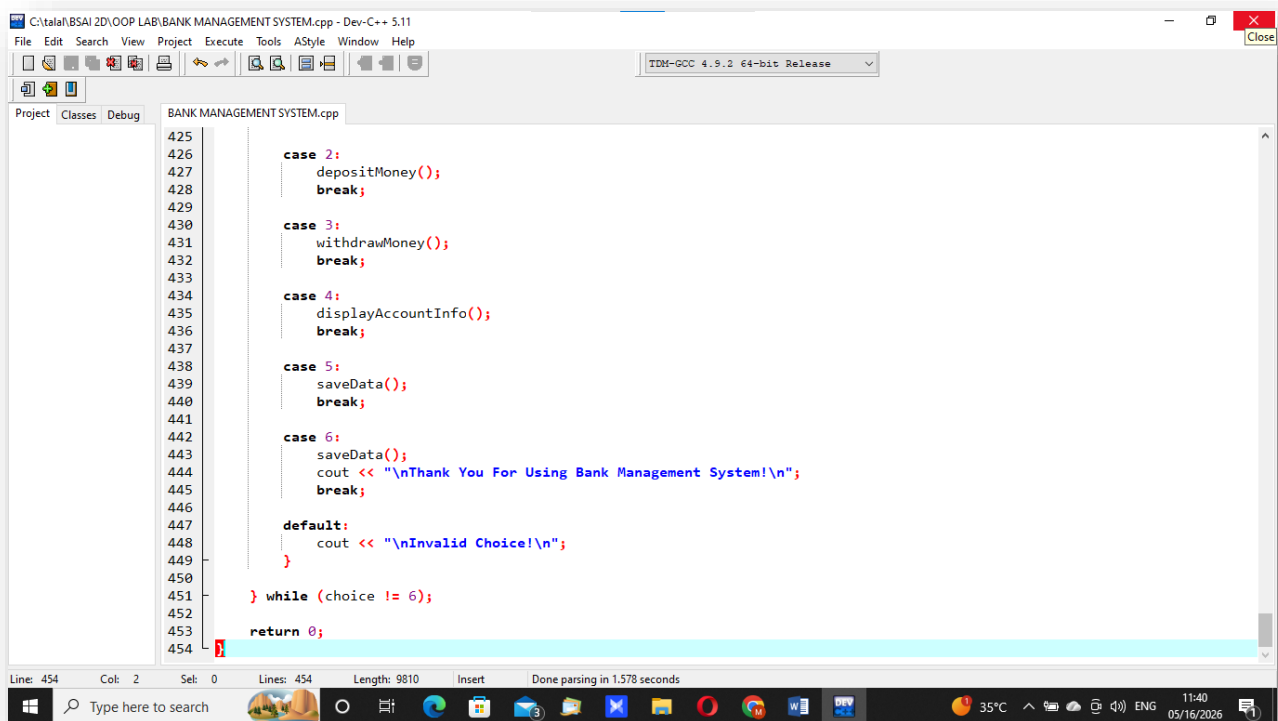
```
380 |         accounts.push_back(s);
381 |     }
382 |     else if (type == "Current")
383 |     {
384 |         CurrentAccount* c = new CurrentAccount();
385 |         c->loadFromFile(file);
386 |
387 |         accounts.push_back(c);
388 |     }
389 |
390 |     file.ignore();
391 | }
392 |
393 | file.close();
394 | }
395 |
396 | // =====
397 | //          MAIN FUNCTION
398 | // =====
399 |
400 | int main()
401 | {
402 |     loadData();
403 |
404 |     int choice;
405 |
406 |     do
407 |     {
408 |         cout << "\n=====
409 |         cout << "\n          BANK MANAGEMENT SYSTEM";
```

ShaheedZulfikarAliBhuttoInstituteofScience&TechnologyUniversity



```
C:\tala\BSAI 2D\OOP LAB\BANK MANAGEMENT SYSTEM.cpp - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release

Project Classes Debug
BANK MANAGEMENT SYSTEM.cpp
410 cout << "\n===== ";
411 cout << "\n1. Create New Account";
412 cout << "\n2. Deposit Money";
413 cout << "\n3. Withdraw Money";
414 cout << "\n4. Check Account Details";
415 cout << "\n5. Save Data";
416 cout << "\n6. Exit";
417 cout << "\nEnter Your Choice: ";
418 cin >> choice;
419
420 switch (choice)
421 {
422 case 1:
423     createNewAccount();
424     break;
425
426 case 2:
427     depositMoney();
428     break;
429
430 case 3:
431     withdrawMoney();
432     break;
433
434 case 4:
435     displayAccountInfo();
436     break;
437
438 case 5:
439     saveData();
440 }
```



```
C:\tala\BSAI 2D\OOP LAB\BANK MANAGEMENT SYSTEM.cpp - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release

Project Classes Debug
BANK MANAGEMENT SYSTEM.cpp
425
426 case 2:
427     depositMoney();
428     break;
429
430 case 3:
431     withdrawMoney();
432     break;
433
434 case 4:
435     displayAccountInfo();
436     break;
437
438 case 5:
439     saveData();
440     break;
441
442 case 6:
443     saveData();
444     cout << "\nThank You For Using Bank Management System!\n";
445     break;
446
447 default:
448     cout << "\nInvalid Choice!\n";
449 }
450
451 } while (choice != 6);
452
453 return 0;
454 }
```

OUTPUT:

```
C:\tala\BSAI 2D\OOP LAB\BANK MANAGEMENT SYSTEM.exe

=====
BANK MANAGEMENT SYSTEM
=====
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Check Account Details
5. Save Data
6. Exit
Enter Your Choice: 1

1. Saving Account
2. Current Account
Choose Account Type: 1

Enter Account Number: 1234
Enter Account Holder Name: talal
Enter Initial Balance: 5000
Enter Interest Rate (%): 5

Account Created Successfully!

=====
BANK MANAGEMENT SYSTEM
=====
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Check Account Details
5. Save Data
6. Exit
Enter Your Choice: 6

Data Saved Successfully!

Thank You For Using Bank Management System!

-----
Process exited after 281.4 seconds with return value 0
Press any key to continue . . .
```

1. **Advantages**

- Demonstrates real-world OOP implementation
- Easy to understand and expand
- Improves programming and problem-solving skills
- Secure data handling using encapsulation

2. **Disadvantages**

- Console-based (no graphical interface)
- Limited to basic banking operations
- No online or network-based functionality

3. **TestingStrategy**

- **Unit Testing:** Test each class and function individually
- **Integration Testing:** Ensure all modules work together
- **Error Testing:** Handle invalid inputs and file errors

4. **Limitations**

- No database integration (uses file handling only)
- Limited security features
- No multi-user support
- Basic interface without GUI

Bank Management System in C++ (Object-Oriented Approach)

Expected Outcomes:

- A fully functional console-based Bank Management System developed in C++ using Object-Oriented Programming (OOP).
- Practical implementation of OOP concepts — classes, objects, encapsulation, constructors, and member functions.
- Improved understanding of program structure using classes and modular design.
- Ability to store and manage customer and account data using file handling.
- Enhanced problem-solving and coding skills through real-world banking application scenarios.
- A simple, interactive platform for managing bank accounts (deposit, withdraw, balance check).
- A solid foundation for future enhancements such as GUI-based or online banking systems.

Future Enhancements:

Future Feature	Related OOP Concept
Add transaction history tracking	Class Functions & File Handling
Add different account types (Saving/Current)	Inheritance & Polymorphism
Store customer records persistently	File Handling with Classes
Use dynamic memory for account storage	Pointers & Dynamic Memory Allocation
Add customer profiles using class objects	Classes & Object Arrays
Implement secure login system	Encapsulation & Data Hiding
Display top customers based on balance	File Streams & Class Objects
Implement exit and restart menu	Control Flow & Member Functions

CONTRIBUTORS:

- ❖ M.TalalKhalid
- ❖ M.RehanKhan
- ❖ Ahmad Raza

Conclusion:

The Bank Management System Project demonstrates the integration of core Object-Oriented Programming concepts in C++ within a real-world application. It helps students understand how classes, objects, encapsulation, and file handling are used to manage banking operations such as account creation, deposits, withdrawals, and balance inquiries.

This project provides practical experience in designing structured and modular programs. It also builds a strong foundation for developing more advanced systems such as GUI-based banking applications and online banking platforms.

References:

1. Deitel & Deitel, C++: How to Program, Pearson Education.
2. Bjarne Stroustrup, The C++ Programming Language, Addison-Wesley.
3. W3Schools — <https://www.w3schools.com/cpp/>
4. GeeksforGeeks — <https://www.geeksforgeeks.org/c-plus-plus/>
5. TutorialsPoint — <https://www.tutorialspoint.com/cplusplus/>